

ASSIGNMENT 1 — NEURAL NETWORKS

Effect of Activation Functions (ReLU, Sigmoid, Tanh) on a Feedforward Network

Problem (iv): Wine Quality Prediction

1. Problem Statement

Experiment with different activation functions (ReLU, Sigmoid, Tanh) in the hidden layers of a feedforward neural network. Evaluate how each activation function affects the network's convergence rate, training time, and final accuracy. Based on the results, discuss which activation function is most suitable for the Wine Quality Prediction problem, and why.

2. Objective

- Build one feedforward network for wine-quality prediction and change only the hidden-layer activation function.
- Compare ReLU, Sigmoid and Tanh on three criteria: convergence rate, training time and final accuracy.
- Recommend the most suitable activation function and justify it using the measured results and theory.

3. Dataset and Task

The **Wine Quality (red wine)** dataset from the UCI Machine Learning Repository is used. It contains **1,599 samples** and **11 physicochemical input features**: fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates and alcohol. The target is a sensory quality score (3–8) given by wine tasters.

Because the score classes are heavily imbalanced (most wines are rated 5 or 6), the task is framed as **binary classification**: a wine is labelled **Good** if quality ≥ 6 and **Not good** otherwise. This gives a fairly balanced split (about 53% Good / 47% Not good), which makes accuracy a meaningful metric.

4. Methodology

4.1 Data preparation

- Stratified split: 64% training, 16% validation, 20% test (fixed random seed 42).
- Features standardised (zero mean, unit variance) using statistics from the training set only, to avoid data leakage.

4.2 Network and training configuration

Setting	Value
Architecture	11 inputs $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 1 output (sigmoid)
Hidden activation	ReLU / Sigmoid / Tanh (the only variable changed)
Loss function	Binary cross-entropy (log-loss)
Optimiser	SGD with momentum 0.9, learning rate 0.01
Batch size / Epochs	32 / 100
Repetitions	10 runs per activation, different weight-initialisation seeds; mean $\pm$ std reported
Implementation	Python, scikit-learn MLPClassifier

4.3 Metrics

- **Convergence rate:** training loss at fixed epochs, and the number of epochs needed to reach a training loss ≤ 0.50 ; also the epoch at which validation loss is lowest.
- **Training time:** wall-clock time spent inside the 100 training epochs (evaluation time excluded).
- **Final accuracy:** test accuracy, reported at the best-validation epoch (early stopping) and at epoch 100.

5. Theory in Brief

Function	Formula	Output range	Derivative (max)
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	1 (for $x > 0$)
Sigmoid	$f(x) = 1 / (1 + e^{(-x)})$	$(0, 1)$	0.25
Tanh	$f(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$	$(-1, 1)$	1.0

- **ReLU:** cheap to compute and does not saturate for positive inputs, so gradients flow well. Neurons with negative input output zero ("dying ReLU" risk) and the network can overfit quickly.
- **Sigmoid:** saturates at both ends and its derivative never exceeds 0.25, so gradients shrink layer by layer (vanishing gradient). Outputs are not zero-centred, which slows learning.
- **Tanh:** zero-centred and with a steeper gradient than sigmoid, so it trains better than sigmoid, but it still saturates for large $|x|$.

6. Results

6.1 Convergence rate

Activation	Loss @ epoch 10	Loss @ epoch 25	Epochs to reach loss ≤ 0.50	Best-validation epoch
ReLU	0.468	0.428	4.4 ± 0.9	18.3 ± 12.5
Sigmoid	0.678	0.566	Not reached (100)	61.2 ± 5.5
Tanh	0.490	0.460	6.0 ± 1.3	41.1 ± 9.8

Table 1: Convergence measures (mean $\pm$ std over 10 runs)

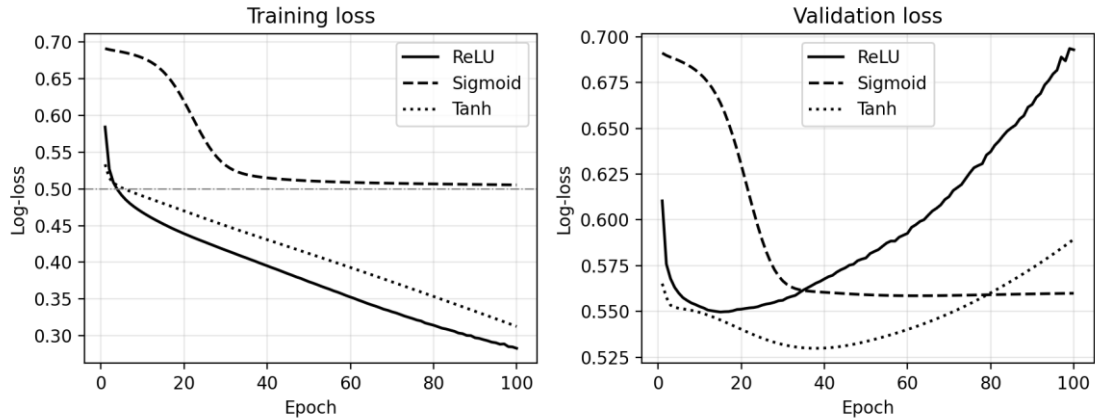


Figure 1: Mean training loss (left) and validation loss (right) over 100 epochs

6.2 Accuracy and training time

Activation	Train acc. (epoch 100)	Min. validation loss	Test acc. (best-val. epoch)	Test acc. (epoch 100)	Training time (100 epochs)
ReLU	88.24%	0.547	$74.47\% \pm 0.90$	74.97%	0.67 s
Sigmoid	76.13%	0.558	$74.53\% \pm 0.53$	74.16%	0.66 s
Tanh	86.65%	0.528	$75.06\% \pm 1.06$	74.91%	0.63 s

Table 2: Accuracy, validation loss and training time (mean over 10 runs)

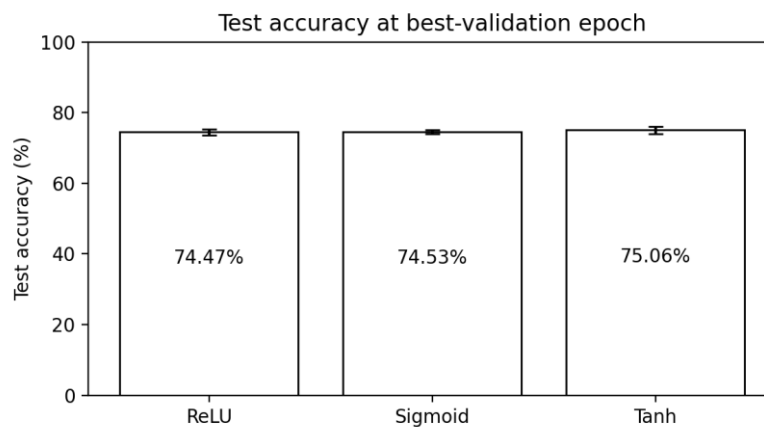


Figure 2: Test accuracy at the best-validation epoch (error bars = 1 std)

7. Analysis and Discussion

Convergence rate. ReLU converged fastest. It reached a training loss of 0.50 in about 4.4 epochs and Tanh took about 6.0 epochs. Sigmoid never reached 0.50 within 100 epochs: it stayed near 0.69 (the

loss of random guessing) for the first ~10 epochs, a clear sign of slow gradient flow, and then levelled off at about 0.505.

Training time. Time per epoch was almost identical (0.63–0.66 s for 100 epochs). The network is small, so the cost of computing the activation is negligible. The practical difference lies in how many epochs are needed, not in the time per epoch: since ReLU needs far fewer epochs to reach a given loss, it is cheaper to train in total. In theory ReLU is also the least expensive function (no exponentials), and this advantage grows for larger networks.

Final accuracy. All three functions ended within about one percentage point of each other on the test set (Tanh 75.06%, Sigmoid 74.53%, ReLU 74.47%). This gap is smaller than the run-to-run standard deviation (about $\pm 1\%$), so accuracy alone does not separate them. Tanh had the lowest validation loss (0.528 versus 0.547 for ReLU and 0.558 for Sigmoid). This is a modest gain that is only slightly larger than the run-to-run spread.

Overfitting behaviour. ReLU fits the training data most aggressively (training accuracy 88.24%), and its validation loss starts rising after roughly epoch 18. Tanh overfits later (about epoch 41). Sigmoid barely overfits, but only because it underfits (training accuracy 76.13%). With ReLU, early stopping or regularisation (dropout, weight decay) is therefore important.

8. Conclusion

ReLU is the most suitable activation function overall for this problem, provided it is used with early stopping. It converges quickly, it is the cheapest to compute, it avoids the vanishing-gradient problem, and it reaches test accuracy (about 74–75%) statistically equal to the other functions. Sigmoid is the least suitable for hidden layers: it converges slowly, plateaus at a higher loss and gains nothing in accuracy. Tanh is a close second, and on this small dataset it gave slightly better validation loss and later overfitting, so it is a reasonable choice when only a shallow network is used and training speed is not critical.

Limitations: the results are from one dataset (red wine), one architecture and one learning rate. Sigmoid could improve with a larger learning rate or more epochs, and differences may grow for deeper networks, where the vanishing-gradient problem of Sigmoid and Tanh becomes more severe.

Appendix: Source Code

File: wine_activation_experiment.py. **Requirements:** Python 3, pandas, numpy, scikit-learn, matplotlib. **How to run:** place winequality-red.csv (semicolon-separated UCI file) in the same folder and run `python wine_activation_experiment.py`

```
"""
Effect of activation functions (ReLU, Sigmoid, Tanh) on a feedforward
neural network -- Wine Quality prediction.

Compares convergence rate, training time and final accuracy.
Dataset : winequality-red.csv (UCI Wine Quality, ';' separated)
Outputs : res.json, loss.png, acc.png
"""

import json
import time
import warnings

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, log_loss
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")

# ----- settings
CSV_FILE = "winequality-red.csv"
ACTIVATIONS = {"ReLU": "relu", "Sigmoid": "logistic", "Tanh": "tanh"}
EPOCHS = 100          # training epochs per run
N_RUNS = 10           # repetitions with different weight initialisation
LOSS_THRESHOLD = 0.50 # used to measure convergence speed
SEED = 42

# ----- data loading
df = pd.read_csv(CSV_FILE, sep=";")
X = df.drop(columns="quality").values
y = (df["quality"] >= 6).astype(int).values # 1 = good wine, 0 = not good

# 64% train / 16% validation / 20% test (stratified)
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.2, random_state=SEED,
    stratify=y_train_val)

# standardise using training statistics only (avoids data leakage)
scaler = StandardScaler().fit(X_train)
X_train, X_val, X_test = (scaler.transform(a) for a in (X_train, X_val, X_test))

# ----- experiment
def run_once(activation, seed):
    """Train one network for EPOCHS epochs and record per-epoch metrics."""
    model = MLPClassifier(
        hidden_layer_sizes=(32, 16),
        activation=activation,
```

```

        solver="sgd",
        learning_rate_init=0.01,
        momentum=0.9,
        batch_size=32,
        random_state=seed,
    )
    train_loss, val_loss, test_acc = [], [], []
    train_time = 0.0

    for _ in range(EPOCHS):
        start = time.time()
        model.partial_fit(X_train, y_train, classes=[0, 1]) # one epoch
        train_time += time.time() - start # evaluation not timed

        train_loss.append(log_loss(y_train, model.predict_proba(X_train)))
        val_loss.append(log_loss(y_val, model.predict_proba(X_val)))
        test_acc.append(accuracy_score(y_test, model.predict(X_test)))

    train_loss, val_loss, test_acc = map(np.array,
                                         (train_loss, val_loss, test_acc))
    best = int(np.argmin(val_loss)) # early-stopping epoch
    reached = np.where(train_loss <= LOSS_THRESHOLD)[0]

    metrics = {
        "ep_th": reached[0] + 1 if len(reached) else np.nan,
        "best_ep": best + 1,
        "time": train_time,
        "tr100": accuracy_score(y_train, model.predict(X_train)),
        "te100": test_acc[-1],
        "te_best": test_acc[best],
        "tl100": train_loss[-1],
        "vlmin": val_loss[best],
        "reach": len(reached) > 0,
    }
    return metrics, train_loss, val_loss

results, curves = {}, {}
for name, act in ACTIVATIONS.items():
    runs, train_curves, val_curves = [], [], []
    for seed in range(N_RUNS):
        m, tl, vl = run_once(act, seed)
        runs.append(m)
        train_curves.append(tl)
        val_curves.append(vl)

    curves[name] = (np.mean(train_curves, axis=0), np.mean(val_curves, axis=0))
    results[name] = {
        key: (float(np.nanmean([r[key] for r in runs])),
              float(np.nanstd([r[key] for r in runs])))
        for key in runs[0]
    }
    results[name]["n_reach"] = int(sum(r["reach"] for r in runs))
    results[name]["tl_ep10"] = float(np.mean([c[9] for c in train_curves]))
    results[name]["tl_ep25"] = float(np.mean([c[24] for c in train_curves]))

# ----- reporting
clean = json.loads(json.dumps(results).replace("NaN", "null"))
with open("res.json", "w") as f:
    json.dump(clean, f, indent=1)

print(f"{'Activation':<10} {'Epochs to loss<=0.5':>20} {'Min val loss':>13} "
      f"{'Test acc':>9} {'Time (s)':>9}")
for name, r in results.items():
    epochs_to = "not reached" if r["n_reach"] == 0 else f"{r['ep_th'][0]:.1f}"
    print(f"{name:<10} {epochs_to:>20} {r['vlmin'][0]:>13.3f} "
          f"{r['te_best'][0] * 100:>8.2f}% {r['time'][0]:>9.2f}")

```

```

# ----- plots
styles = {"ReLU": "-", "Sigmoid": "--", "Tanh": ":"}

fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
for name, (train_c, val_c) in curves.items():
    axes[0].plot(range(1, EPOCHS + 1), train_c, styles[name], color="k",
                  lw=1.6, label=name)
    axes[1].plot(range(1, EPOCHS + 1), val_c, styles[name], color="k",
                  lw=1.6, label=name)
for ax, title in zip(axes, ("Training loss", "Validation loss")):
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Log-loss")
    ax.set_title(title)
    ax.grid(alpha=0.3)
    ax.legend()
axes[0].axhline(LOSS_THRESHOLD, color="gray", lw=0.8, ls="-.")
plt.tight_layout()
plt.savefig("loss.png", dpi=200)

names = list(results)
plt.figure(figsize=(6, 3.4))
plt.bar(names, [results[n]["te_best"][0] * 100 for n in names],
        yerr=[results[n]["te_best"][1] * 100 for n in names],
        color="white", edgecolor="k", capsize=4)
for i, n in enumerate(names):
    plt.text(i, results[n]["te_best"][0] * 50,
             f'{results[n]["te_best"][0] * 100:.2f}%', ha="center")
plt.ylim(0, 100)
plt.ylabel("Test accuracy (%)")
plt.title("Test accuracy at best-validation epoch")
plt.tight_layout()
plt.savefig("acc.png", dpi=200)

```